Sisteme de operare

Laborator 7

Semestrul I 2023-2024

Vlad Olaru

Laborator 7

- comunicare inter-procese (IPC)
- POSIX threads
  - mutex-uri
  - semafoare
  - monitoare

POSIX Mutex Locks

- crearea si initializarea lock-ului mutex

```
#include <pthread.h>

pthread_mutex_t mutex;

/* create and initialize the mutex lock */
pthread_mutex_init(&mutex,NULL);
```

- obtinerea si eliberarea lock-ului

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);

/* critical section */

/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

Semafoare POSIX

- cu nume (*named*) sau fara nume (*unnamed*)
- cele cu nume pot fi folosite de procese diferite (neinrudite), cele fara nume nu

Semafoare POSIX cu nume

- creare si initializare:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- alte procese acceseaza semaforul cu ajutorul numelui **SEM**.
- obtinerea si eliberarea semaforului:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

Semafoare POSIX fara nume

- creare si initializare:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

- obtinerea si eliberarea semaforului:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```

Variabile conditie POSIX

- POSIX folosit uzual impreuna cu C/C++
- C/C++ nu ofera suport pentru monitoare la nivel de limbaj

=> POSIX implementeaza monitoare la nivel de biblioteca *pthreads* prin asocierea variabilelor conditie cu mutex-uri

- creare si initializare “monitor” POSIX:

```
pthread_mutex_t mutex;
pthread_cond_t cond_var;

pthread_mutex_init(&mutex,NULL);
pthread_cond_init(&cond_var,NULL);
```

Variabile conditie POSIX (cont.)

- thread care asteapta ca o conditie, **`a == b,`** sa devina adevarata:

```
pthread_mutex_lock(&mutex);
while (a != b)
     pthread_cond_wait(&cond_var, &mutex);

pthread_mutex_unlock(&mutex);
```

- thread care deblocheaza alt thread care asteapta la o variabila conditie:

```
pthread_mutex_lock(&mutex);
a = b;
pthread_cond_signal(&cond_var);
pthread_mutex_unlock(&mutex);
```

Producator-consumator POSIX

```c
#include <stdio.h>
#include <pthread.h>

#define ITEMS           8
#define BUFFER_SIZE     4
int buffer[BUFFER_SIZE];

void *producer(void*);
void *consumer(void*);

int spaces, items, tail;
pthread_cond_t space, item;
pthread_mutex_t buffer_mutex;

int main()
{
        pthread_t producer_thread, consumer_thread;
        void *thread_return;
        int result;

        spaces = BUFFER_SIZE;
        items = 0;
        tail = 0;

        pthread_mutex_init(&buffer_mutex, NULL);
        pthread_cond_init(&space, NULL);
        pthread_cond_init(&item, NULL);

        if(pthread_create(&producer_thread, NULL, producer, NULL) ||
           pthread_create(&consumer_thread, NULL, consumer, NULL))
                exit(1);

        if(pthread_join(producer_thread, &thread_return))
                exit(1);
        else
                printf("producer returns with %d\n", (int)thread_return);

        if(pthread_join(consumer_thread, &thread_return))
                exit(1);
        else
                printf("consumer returns with %d\n", (int)thread_return);
        exit(0);
}

```